Aurora 프로토콜 이론적 배경

개요

Aurora는 Ben-Sasson, Chiesa, Riabzev, Spooner, Virza, Ward가 2019년에 제안한 투명한 zkSNARK 프로토콜입니다. Aurora의 핵심 혁신은 FRI(Fast Reed-Solomon Interactive Oracle Proof) 프로토콜을 활용하여 $O(\log^2 N)$ 크기의 인수(argument)를 달성한 것입니다. 이는 Ligero의 O(N^0.5)보다 점근적으로 우수한 성능을 제공합니다.

Aurora의 핵심 혁신

1. FRI 기반 저차 테스트 (Low-Degree Test)

Aurora의 가장 중요한 혁신은 FRI 프로토콜을 사용한 효율적인 저차 테스트입니다:

- **기존 방식의 한계**: Ligero는 직접적인 근접성 테스트를 사용하여 O(N^0.5) 쿼리 복잡도를 가집니다
- **FRI의 장점**: 재귀적 구조를 통해 O(log N) 쿼리 복잡도를 달성합니다
- **점근적 개선**: 대규모 계산에서 Ligero 대비 현저한 성능 향상을 제공합니다

2. $O(\log^2 N)$ 인수 크기 달성 방법

Aurora가 $O(\log^2 N)$ 인수 크기를 달성하는 핵심 메커니즘:

2.1 FRI의 재귀적 구조

```
초기 다항식 f₀ (차수 d₀)

↓ 폴딩 (folding)
```

```
다항식 f₁  (차수 d₁  ≈ d₀/2)

↓ 폴딩

다항식 f₂  (차수 d₂  ≈ d₁ /2)

⋮

다항식 fₖ (차수 dₖ < 상수)
```

2.2 각 라운드의 기여도

- **라운드 수**: O(log N) (다항식 차수가 절반씩 감소)
- **라운드당 커밋먼트**: O(log N) (머클 트리 루트)
- **총 인수 크기**: O(log N) × O(log N) = $O(\log^2 N)$

3. 투명성과 양자 후 보안성

Aurora는 다음과 같은 보안 특성을 제공합니다:

- **투명성**: 신뢰할 수 있는 설정(trusted setup) 불필요
- **양자 후 보안성**: 해시 함수의 보안성에만 의존
- **사운드니스**: 통계적 보안성 제공 (계산적 가정 불필요)

FRI (Fast Reed-Solomon Interactive Oracle Proof) 프로토콜의 원리

1. FRI의 기본 아이디어

FRI는 주어진 함수가 저차 다항식에 가까운지 효율적으로 검증하는 프로토콜입니다:

1.1 폴딩 (Folding) 기법

다항식 f(x)에 대해 다음과 같은 변환을 수행합니다:

```
f(x) = f_even(x²) + x · f_odd(x²)
```

여기서:

- f_even(x^2): f(x)의 짝수 차수 항들로 구성된 다항식
- f_odd(x^2): f(x)의 홀수 차수 항들로 구성된 다항식

1.2 재귀적 검증

```
f가 차수 d의 다항식 ⇔ f_folded가 차수 ≤ d/2의 다항식
```

이 성질을 이용하여 재귀적으로 차수를 줄여가며 검증합니다.

2. FRI의 단계별 동작 과정

2.1 커밋 단계 (Commit Phase)

```
// libiop/protocols/ldt/fri/fri_ldt.hpp에서 발췌
template<typename FieldT>
class FRI_protocol : public multi_LDT_base<FieldT> {
std::vector<field_subset<FieldT> > domains_;
std::vector<localizer_polynomial<FieldT> > localizer_polynomials_;
std::vector<std::vector<std::vector<oracle_handle_ptr> > > oracle_handles_;
};
```

1. **초기 다항식 커밋**: 프로버가 f_0를 머클 트리로 커밋
2. **폴딩 라운드**: 각 라운드에서 $f_{i+1} = \text{fold}(f_i, \alpha_i)$ 계산 및 커밋

3. **최종 다항식**: 충분히 작은 차수가 될 때까지 반복

2.2 쿼리 단계 (Query Phase)

1. **랜덤 위치 선택**: 검증자가 랜덤하게 쿼리 위치들 선택
2. **일관성 검증**: 각 폴딩 단계에서의 일관성 확인
3. **최종 검증**: 마지막 다항식의 차수 확인

3. FRI의 보안성 분석

3.1 사운드니스 (Soundness)

- **거리 기반 분석**: Reed-Solomon 코드의 최소 거리 활용
- **폴딩 보존성**: 각 폴딩 단계에서 거리 보존 증명
- **확률적 검증**: 높은 확률로 저차가 아닌 함수 탐지

3.2 완전성 (Completeness)

- **정직한 프로버**: 실제 저차 다항식에 대해 항상 검증 통과
- **효율적 증명**: 다항식 시간 내 증명 생성 가능

다중 라운드 구조와 라운드별 역할 분석

1. Aurora의 전체 구조

Aurora는 다음과 같은 계층적 구조를 가집니다:

```
Aurora zkSNARK
├── R1CS-to-IOP 변환 (encoded_aurora_protocol)
├── FRI 저차 테스트 (FRI_protocol)
```

```
└── BCS 변환 (Fiat-Shamir)
```

2. 라운드별 상세 분석

2.1 라운드 1: 증인 오라클 제출

```
// aurora_iop.hpp에서 발췌
void produce_proof(const r1cs_primary_input<FieldT> &primary_input,
const r1cs_auxiliary_input<FieldT> &auxiliary_input);
```

목적: R1CS 증인을 다항식 형태로 인코딩하여 제출

주요 동작:

- 변수 할당을 다항식으로 변환
- Reed-Solomon 인코딩 적용
- 머클 트리로 커밋

2.2 라운드 2: 제약 조건 검증

목적: R1CS 제약 조건의 만족 여부 확인

주요 동작:

- 제약 조건 다항식 계산
- 영지식성을 위한 마스킹 (make_zk = true인 경우)
- 합산 검사 (sumcheck) 수행

2.3 라운드 3-k: FRI 저차 테스트

목적: 제출된 다항식들이 실제로 저차인지 검증

각 FRI 라운드의 역할:

1. **폴딩 계수 수신**: 검증자로부터 랜덤 α 수신
2. **다항식 폴딩**: $f_{i+1}(x) = f_i_even(x) + \alpha \cdot f_i_odd(x)$
3. **새로운 커밋**: 폴딩된 다항식을 머클 트리로 커밋

2.4 최종 라운드: 쿼리 응답

목적: 검증자의 쿼리에 대한 일관성 있는 응답 제공

주요 동작:

- 쿼리 위치에서의 다항식 값 제공
- 머클 증명 (authentication path) 제공
- 폴딩 일관성 검증을 위한 보조 정보 제공

3. 라운드 복잡도 분석

3.1 이론적 복잡도

- **총 라운드 수**: O(log N)
- **각 라운드 통신량**: O(log N) (머클 루트)
- **총 통신 복잡도**: $O(\log^2 N)$

3.2 실제 구현에서의 최적화

```
// FRI_protocol_parameters에서 발췌
std::vector<size_t> localization_parameters_;
size_t num_interactive_repetitions_;
size_t num_query_repetitions_;
```

지역화 매개변수 (Localization Parameters):

- 각 라운드에서 차수 감소 비율 조정
- 증명 크기와 보안성의 트레이드오프 제어

반복 매개변수 (Repetition Parameters):

- 보안 수준에 따른 반복 횟수 결정
- 사운드니스 오류 확률 제어

성능 특성 및 장점

1. 점근적 성능

- **인수 크기**: $O(\log^2 N)$
- **검증 시간**: $O(\log N)$
- **증명 시간**: $O(N \log N)$

2. 실용적 장점

- **확장성**: 대규모 계산에 적합
- **투명성**: 신뢰할 수 있는 설정 불필요
- **양자 안전성**: 해시 기반 보안성

3. Ligero와의 비교

| 특성 | Aurora | Ligero |

|------|--------|--------|

| 인수 크기 | $O(\log^2 N)$ | O(N^0.5) |

| 라운드 수 | O(log N) | 2 |

| 쿼리 복잡도 | O(log N) | O(N^0.5) |

| 증명 시간 | O(N log N) | O(N) |

Aurora는 특히 대규모 계산 ($N > 2^{20}$)에서 Ligero보다 우수한 성능을 보입니다.

결론

Aurora 프로토콜은 FRI를 활용한 혁신적인 접근 방식으로 투명한 zkSNARK의 새로운 가능성을 제시했습니다. $O(\log^2 N)$의 점근적 성능과 실용적인 구현 가능성을 동시에 제공하여, 대규모 영지식 증명 시스템의 기반 기술로 자리잡았습니다. 특히 FRI의 재귀적 구조는 이후 STARK 계열 프로토콜들의 핵심 구성 요소가 되었습니다.